

Mikrocomputertechnik 1

Einheit 3: Speicherzugriff & Kontrollfluss

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 72

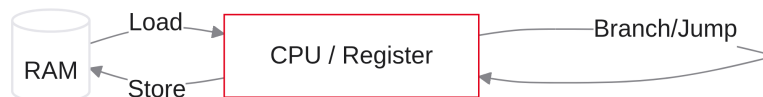
Speicherzugriff & Kontrollfluss

Folie

2 / 72

Von Arrays zu Schleifen

- Herzlich willkommen zur dritten Vorlesung in Mikrocomputertechnik 1
- Heute: von einfachen Variablen zu großen Datenmengen
- Kommunikation der CPU mit dem Arbeitsspeicher (RAM)
- Ausbruch aus dem linearen Programmablauf: Entscheidungen treffen
- Ziel: C-Schleifen in Hardware-nahen Assembler-Code übersetzen

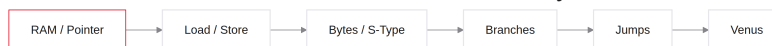


Notizen

Herzlich willkommen zu Einheit 3. Bisher haben wir uns in einer sicheren, aber kleinen Welt bewegt: der Werkbank mit 32 Registern. Addieren und Subtrahieren funktionieren, solange alle Daten griffbereit sind. Ein Bild mit zwei Millionen Pixeln passt dort nicht hinein. Heute: Daten aus dem Arbeitsspeicher holen, verarbeiten und zurücklegen — und dem Prozessor beibringen, If-/Else-Entscheidungen und Schleifen auszuführen.

Agenda für den heutigen Tag

1. Der Arbeitsspeicher — RAM-Konzept und Pointer-Grundlagen
2. Load & Store (Words) — 32-Bit-Daten und die 4-Byte-Schrittweite
3. Teilzugriffe — einzelne Bytes und das S-Type-Format
4. Kontrollfluss — bedingte Sprünge (Branches) für If-Abfragen
5. Unbedingte Sprünge — Jumps und das Überspringen von Else-Blöcken
6. Praxis in Venus — For- und While-Schleifen über Arrays



Notizen

Die Route teilt sich in zwei Hälften. Erstens die Daten: Speichermodell aus Einheit 1, Load-/Store-Befehle und Pointer-Arithmetik in Assembler. Zweitens die Logik: den Program Counter manipulieren, damit Code dynamisch springt. Zum Schluss führen wir beides im Venus-Simulator zusammen.

Rückblick: Was wir bisher können

- Basis-Arithmetik: `add`, `sub` und logische Operationen
- Strikte Assembler-Syntax: Befehl `rd, rs1, rs2`
- I-Type: Konstanten direkt im Code (`addi`)
- Wichtige ABI-Register: `t0–t6`, `a0–a7`, `s0–s11`
- Jeder Befehl ist intern exakt 32 Bit groß

```
# Bisheriges Maximum an Komplexität:  
li t0, 100      # Lade Konstante 100  
addi t1, t0, 50 # t1 = 100 + 50
```

Notizen

Kurz rekapituliert: Im letzten Labor haben Sie Register addiert, Bits verschoben und mit Konstanten gerechnet. Pseudo-Instruktionen wie `li` nehmen Arbeit ab. Gemeinsamkeit aller bisherigen Operationen: Sie wirken nur auf Daten, die fest im Code stehen oder bereits in der CPU liegen. Wir waren isoliert vom Rest des Computers.

Das Register-Limit (Die Werkbank)

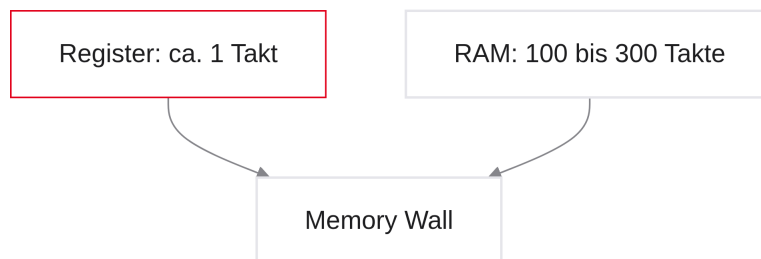
- Die CPU besitzt exakt 32 Architektur-Register (unsere „Werkbank“)
- Jedes Register fasst exakt 32 Bit (4 Bytes)
- Gesamte Arbeitskapazität der CPU: nur 128 Bytes
- Ein C-Array mit 1000 Integern benötigt bereits 4000 Bytes
- Echte Programme und Datenmassen passen nie komplett in die CPU

Notizen

Zentrale Erkenntnis: Das Register-File ist extrem schnell, aber winzig — $32 \times 4 = 128$ Bytes. Das reicht kaum für einen kurzen Text. Sensordaten oder Bilder brauchen den Main Memory (RAM) mit vielen Gigabytes. Hardwarenahe Programmierung heißt: Daten geschickt vom großen, langsameren Lager auf die kleine, schnelle Werkbank jonglieren.

Die Memory Wall (Latenzen)

- Daten in Registern: typisch in etwa einem Taktzyklus verfügbar
- Der Main Memory (RAM) liegt physisch außerhalb der CPU
- Ein RAM-Zugriff dauert oft 100 bis 300 Taktzyklen
- Diese Geschwindigkeitskluft heißt Memory Wall
- Jeder unnötige RAM-Zugriff bremst das Programm stark ab



Notizen

Warum nicht einfach mehr Register? Physik setzt Grenzen. Der Arbeitsspeicher liegt Zentimeter entfernt; Signale brauchen Zeit. Register: ein Takt. RAM: oft 100–300 Takte — der Prozessor wartet (Stalls). Deshalb halten gute Compiler Variablen möglichst lange in Registern und schreiben selten in den RAM.

Das RAM-Modell (Refresher)

- RAM als langes Array aus winzigen Fächern
- Jedes Fach hat eine eindeutige Hausnummer: die Speicheradresse
- Adressen in RV32I: 32 Bit (0×00000000 bis $0 \times \text{FFFFFFFF}$)
- Ein 32-Bit-Pointer adressiert 2^{32} Bytes — also 4 Gigabyte
- Das C-Programm sieht diesen linearen Adressraum für sich allein

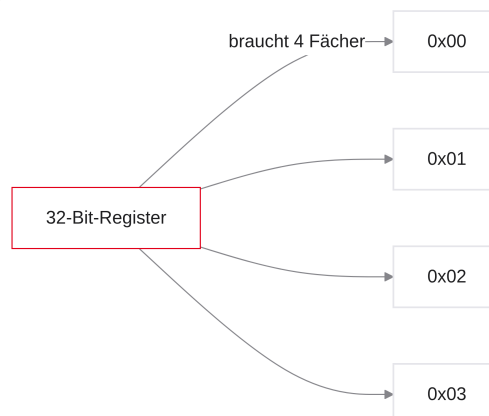
Adresse (Hex)	Wert (Hex)	Bedeutung
0×10000000	0×12	Erstes Fach
0×10000001	$0 \times \text{AB}$	Zweites Fach
0×10000002	0×00	Drittes Fach

Notizen

Erinnerung an Einheit 1: Der Arbeitsspeicher ist logisch simpel — ein endloses Regal mit Hausnummern (Adressen). Mit 32-Bit-Adressen sind maximal 2^{32} Fächer möglich, also 4 Gigabyte. „Lies Fach 0×10000000 “ — der Memory-Controller adressiert genau diesen Ort im RAM.

Byte-Adressierbarkeit

- Jedes Fach im RAM fasst exakt 1 Byte (8 Bits)
- Der Arbeitsspeicher ist byte-adressierbar
- Ein RISC-V-Register fasst 32 Bits (4 Bytes = 1 Word)
- Ein Word laden heißt: vier aufeinanderfolgende Fächer lesen
- Ein 32-Bit-Array belegt daher die Adressen 0, 4, 8, 12, ...



Notizen

Wichtiger Punkt: Der Speicher ist in Bytes aufgeteilt, nicht in Wörter. Ein Fach fasst 8 Bits; die CPU rechnet mit 32 Bits. Ein Integer wird in vier 8-Bit-Stücke zerlegt und belegt die Adressen 0, 1, 2 und 3 — die nächste freie Startadresse ist 4. Das ist die Grundlage der Pointer-Arithmetik.

Folie

9/72

Endianness (Refresher)

- In welcher Reihenfolge landen vier Bytes einer 32-Bit-Zahl im RAM?
- RISC-V nutzt standardmäßig Little-Endian (wie x86)
- Das niederwertigste Byte (LSB) liegt auf der kleinsten Adresse
- Das höchstwertige Byte (MSB) liegt auf der größten Adresse
- Relevant beim Debuggen im Venus-Memory-Tab

```
Zahl 0xAABBCCDD an Adresse 0x100:  
RAM 0x100: DD    (LSB zuerst)  
RAM 0x101: CC  
RAM 0x102: BB  
RAM 0x103: AA
```

Notizen

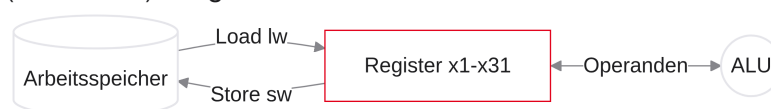
Beim Ablegen von 0xAABBCCDD entscheidet die Byte-Reihenfolge. RISC-V (wie Intel) nutzt Little-Endian: das LSB (DD) kommt auf die kleinste Adresse, danach CC, BB, und das MSB (AA) in das vierte Fach. Im Venus-Speicher wirkt die Zahl optisch „rückwärts“ — das ist erwartetes Little-Endian-Verhalten.

Folie

10/72

Die Load/Store-Architektur

- In Einheit 2: goldene RISC-Regel — die ALU greift nie direkt auf den RAM zu
- Befehle wie `add` mit RAM-Operanden existieren in RISC-V nicht
- Zwei Tore zwischen CPU und RAM:
 - Load (Lesen): RAM → Register
 - Store (Schreiben): Register → RAM



Notizen

Zwei Variablen aus dem RAM addieren: drei Schritte. (1) Load Variable 1 nach `t0`. (2) Load Variable 2 nach `t1`. (3) `add` in der ALU. Die ALU ist vom RAM isoliert — jeder Datenverkehr läuft über Load und

Was ist ein Pointer?

- Ein Pointer ist eine 32-Bit-Hausnummer (Speicheradresse) — keine Magie
- In Assembler liegen Adressen in normalen Registern ($x0-x31$)
- Die CPU unterscheidet die Zahl 10 nicht von der Adresse $0x1000$
- Erst der Befehl (add vs. lw) bestimmt die Interpretation

```
// C: Pointer anlegen
int array[3] = {10, 20, 30};
int *ptr = array; // z. B. 0x00400000

// In Assembler ist ptr nur ein Wert in einem Register
```

Notizen

In C wirken Pointer abstrakt; in Assembler sind sie schlicht 32-Bit-Zahlen. Liegt in `s1` der Wert $0x00400000$, ist das zunächst nur eine Zahl. Durch die ALU: Mathematik. Am Memory-Controller: Hausnummer. Ein Pointer ist eine Adresse, die auf ihren Einsatz wartet.

Adressen in Registern

- Mit `li` (Load Immediate) Pointer initialisieren
- Die Adresse im Register dient als Base-Pointer
- Der Base-Pointer markiert den Start eines Arrays im RAM
- `addi` auf den Pointer entspricht Bewegung im Speicher
- Von diesem Anker aus adressieren wir alle Elemente

```
# Array startet bei 0x10000000 - Base-Pointer in s1:
li s1, 0x10000000

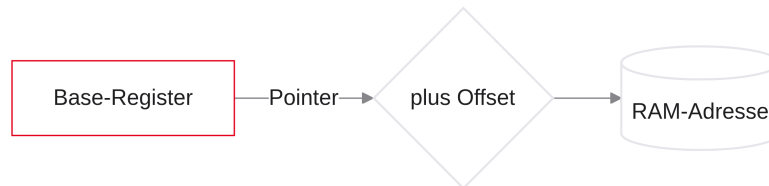
# s1 ist der Finger, der auf den RAM zeigt
```

Notizen

Wie kommt der Pointer ins Register? Oft kennen Compiler oder Betriebssystem die Startadresse. Mit `li` schreiben wir sie z. B. nach `s1` — unserem Base-Pointer, dem Zeigefinger auf Element 0. Verschieben heißt: etwas zu `s1` addieren.

Load & Store (Ganze Wörter)

- Datenfluss zwischen Register und RAM etablieren
- RISC-V-Adressierung: Base + Offset
- `lw` (Load Word): 32-Bit-Blöcke lesen
- `sw` (Store Word): 32-Bit-Blöcke schreiben
- Stolperfalle: 4-Byte-Schrittweite bei Integer-Arrays



Notizen

Als Nächstes: die konkreten Assembler-Befehle. Relative Adressierung (Base + Offset) formuliert Speicherzugriffe elegant. Und wir klären, warum Integer-Arrays in Vierer-Schritten durchlaufen werden.

Das Problem der Adressierung

- Array mit 5 Werten; Base-Pointer `s1` zeigt auf den Start
- Wie greifen wir elegant auf Element 3 zu?
- Schlechte Idee: `addi s1, s1, 12` — Startpunkt geht verloren
- Besser: temporärer Versatz, der den Pointer nicht zerstört
- Lösung: Syntax der Load-/Store-Befehle

Notizen

Zeigt `s1` auf den Array-Start und Sie addieren 8 direkt auf `s1`, zeigt der Finger zwar auf Element 2 — aber Element 0 ist „vergessen“. Wir brauchen einen Mechanismus, der vom Basis-Finger nur temporär greift, ohne den Anker zu verschieben.

Base + Offset (Konzept)

- Speicherzugriffe in RISC-V: immer Base + Offset
- Base: Register mit der Startadresse (z. B. `s1`)
- Offset: konstante Zahl (Immediate), temporär addiert
- Zieladresse = Registerwert + Konstante
- Der Wert im Base-Register bleibt unverändert

```
// C-Metapher für Base + Offset:  
int element = array[3];  
// array = Base, 3 (bzw. 12 Bytes) = Offset
```

Notizen

Base + Offset ist wie eine Wegbeschreibung: Straße X (Base), von dort 12 Meter weiter (Offset). Die Zieladresse wird live berechnet; das Base-Register bleibt Anker. Genau so arbeiten Arrays in C unter der Haube.

Load Word (lw)

- 32-Bit-Wort aus dem RAM in ein Register laden
- Syntax: `lw rd, offset(rs1)`
- `rd`: Zielregister für den RAM-Wert
- `offset`: Konstante (12-Bit-Immediate)
- `rs1`: Base-Register (in Klammern)
- Lädt exakt 4 Bytes ab der berechneten Adresse

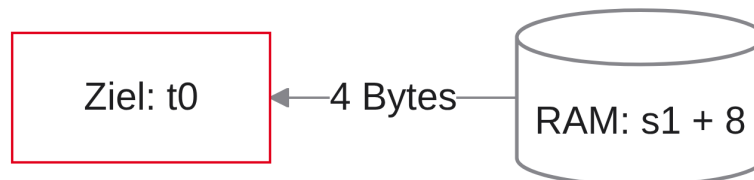
```
# Element 8 Bytes nach dem Pointer in s1 → t0  
lw t0, 8(s1)  
  
# Adresse = s1 + 8; hole 4 Bytes
```

Notizen

`lw` — Load Word. Syntax: Zielregister, dann Offset, dann Base in runden Klammern. Die Klammern signalisieren: Zugriff auf den Arbeitsspeicher. Hardware addiert Offset zu `s1`, liest vier Bytes in das Zielregister.

Datenfluss beim lw

- Flussrichtung visualisieren: wie eine C-Zuweisung
- Bei lw fließt das Datum von rechts nach links
- Klammer (rechts): Quelle (RAM)
- Register (links): Ziel



Notizen

Wie bei R-Type-Befehlen: Ziel steht links. Daten fließen von der rechts berechneten Adresse nach links ins Register — analog zu $t0 = RAM[...]$.

Store Word (sw)

- 32-Bit-Wort vom Register zurück in den RAM schreiben
- Syntax: `sw rs2, offset(rs1)`
- `rs2`: Quellregister (zu speichernde Daten)
- `offset(rs1)`: Zieladresse im RAM
- Ausnahme: Ziel (RAM) steht rechts — Quellregister links

```
# Wert aus t0 nach RAM-Adresse (s1 + 12)
```

```
sw t0, 12(s1)
```

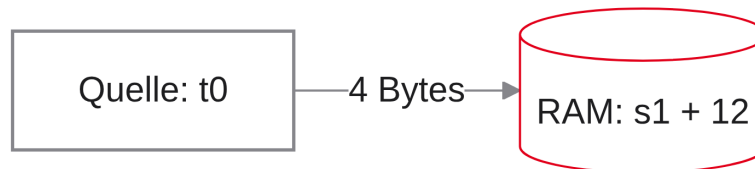
```
# t0 wird nicht überschrieben - es ist die Quelle
```

Notizen

Gegenstück: Store Word (sw). Hier steht links das Quellregister, rechts die Adressberechnung. `t0` wird nicht verändert — nur kopiert. Scheinbarer Bruch der Regel „Ziel links“: das Ziel ist der RAM.

Datenfluss beim `sw`

- Bei `sw` fließt das Datum von links nach rechts
- Register (links): Quelle
- Berechnete RAM-Adresse (rechts): Ziel
- Store schreibt keinen neuen Wert ins Register File — kein `rd`



Notizen

Pfeil umgekehrt: Daten verlassen die CPU und strömen in den Speicher. Store ist der erste Befehl ohne Zielregister `rd`, den wir kennenlernen (Branches haben ebenfalls keines) — wichtig später für die Control Unit.

C-Arrays vs. Assembler

- Array aus 32-Bit-Integern (`int`) durchlaufen
- In C abstrahiert: `array[0]`, `array[1]`, `array[2]`
- In Assembler: Bytes zählen — keine Index-Magie
- Jeder `int` ist 4 Bytes groß → Adressen um 4 erhöhen
- Häufigste Fehlerquelle im Labor

```
// C: Indizes steigen um 1
array[0] = 10;
array[1] = 20;
array[2] = 30;
```

Notizen

In C steigen Indizes um 1. In Assembler fasst jedes RAM-Fach 1 Byte, der Integer aber 4. Offset um 1 erhöhen landet mitten in der Zahl — wir müssen die Pointer-Mathematik selbst machen.

Die 4-Byte-Schrittweite

- Startadressen aufeinanderfolgender 32-Bit-Wörter: Vielfache von 4
- Index 0 → Offset 0
- Index 1 → Offset 4
- Index 2 → Offset 8
- Index 3 → Offset 12

C-Index	Assembler-Offset	RAM (Base = 0x100)
[0]	0	0x100–0x103
[1]	4	0x104–0x107
[2]	8	0x108–0x10B

Notizen

Offsets für Word-Arrays: fast immer 0, 4, 8, 12, 16. Element 0 an 100 belegt 100–103; Element 1 beginnt bei 104. `lw t0, 1(s1)` für „zweites Element“ ist in der Klausur falsch.

Element 0 laden

- Pointer auf Array-Start liegt in `s1`
- `array[0]` nach `t0` laden
- Offset ist 0

```
# Lade array[0]
lw t0, 0(s1)

# RAM-Adresse = s1 + 0
```

Notizen

Einfachstes Szenario: Base zeigt bereits auf Element 0. Offset 0 — `lw t0, 0(s1)`. Die CPU liest die ersten vier Bytes.

Element 1 und 2 laden

- `array[1]` und `array[2]` laden
- Offsets: 4 und 8
- Base-Register `s1` bleibt unverändert (weiterhin Element 0)

```
lw t1, 4(s1)      # array[1], Adresse = s1 + 4
lw t2, 8(s1)      # array[2], Adresse = s1 + 8

# s1 ändert sich durch die Offsets nicht
```

Notizen

Offset 4 und 8 laden die nächsten Elemente, ohne `s1` zu zerstören. Intern berechnet die ALU kurz `s1 + 8`; der Registerwert bleibt Anker.

Die Illusion des C-Compilers

- Warum nie `array[i * 4]` in C schreiben?
- Der Compiler nimmt Ihnen die Skalierung ab
- Bei `int`-Arrays weiß er: Elementgröße 4 Bytes
- Aus `array[i]` wird Assembler mit $i \cdot 4$ vor dem Load
- Assembler enttarnt diese High-Level-Abstraktion

```
// Sie schreiben:
int x = array[i];

// Der Compiler denkt:
// Typ int (4 Byte) → Adresse = &array + (i * 4)
```

Notizen

C abstrahiert die Physik. Der Datentyp `int` signalisiert 4 Bytes; `array[3]` erzeugt unsichtbar die Offset-Rechnung $3 \cdot 4 = 12$. In Assembler gibt es diese Typ-Magie nicht — Sie steuern jedes Byte.

Komplettes Beispiel (Lesen)

- C-Zeile: `a = array[2];`
- `a` in `s0`, Array-Base in `s1`
- Index 2 $\rightarrow$ Offset $2 \cdot 4 = 8$

```
int a = array[2];
```

```
# s0 = a, s1 = Base-Pointer  
lw s0, 8(s1)
```

Notizen

Simple Zuweisung $\rightarrow$ ein Assembler-Befehl: `lw s0, 8(s1)`. Variable `a` (`s0`) erhält den korrekten Wert.

Komplettes Beispiel (Schreiben)

- Modifikation: `array[3] = a + b;`
- Annahmen: `a` in `s0`, `b` in `t0`, Array in `s1`
- Zwei Schritte: erst ALU, dann Store

```
array[3] = a + b;
```

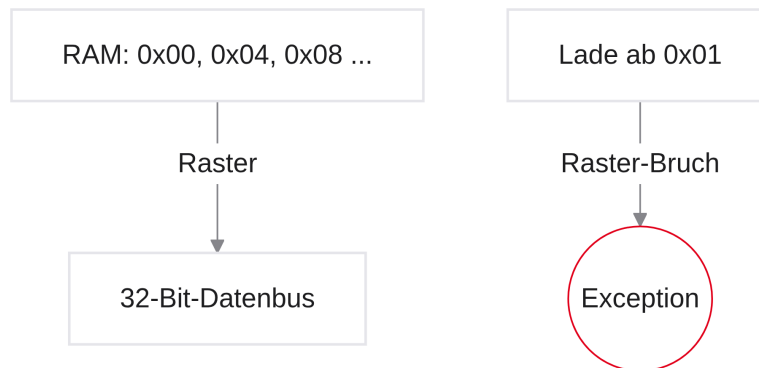
```
add t1, s0, t0      # t1 = a + b  
sw t1, 12(s1)       # Index 3  $\rightarrow$  Offset 12
```

Notizen

Die ALU schreibt nicht direkt in den RAM. Erst `add` nach `t1`, dann `sw` mit Offset 12 für Index 3.

Speicherausrichtung (Alignment)

- 32-Bit-Wörter beginnen auf durch 4 teilbaren Adressen
- Gültig: 0×00 , 0×04 , 0×08 , $0 \times 0C$, ...
- Laden ab 0×01 ? Oft Alignment Fault / Trap — Verhalten implementierungsabhängig
- Der 32-Bit-Datenbus zieht physisch 4-Byte-Blöcke im Raster

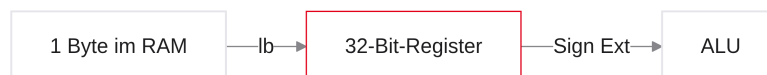


Notizen

Wörter müssen auf 0, 4, 8, 12 ... ausgerichtet sein. Der Bus ist fest verdrahtet; viele Implementierungen trafen bei schiefen Zugriffen, echte Hardware oft strikt — Venus ist dagegen häufig toleranter. Offsets trotzdem sorgfältig wählen.

Teilzugriffe & S-Type-Format

- Nicht alles ist ein 32-Bit-Integer (z. B. ASCII-Zeichen)
- Zugriff auf einzelne Bytes: `lb` und `sb`
- Vorzeichenerweiterung bei kleinen Daten
- Signed vs. Unsigned Loads
- S-Type-Instruction-Format für Store-Befehle



Notizen

Buchstaben in Strings sind 1 Byte groß. Mit `lw` vier Zeichen auf einmal zu laden wäre falsch. Wir brauchen feinere Befehle für Teilbereiche — und später den Blick auf den Maschinencode von Store (kein Zielregister).

Problem: Kleine Daten laden

- Ein C-String (`char[]`) hat Elemente von 1 Byte
- `lw` würde vier Buchstaben gleichzeitig laden
- Lösung: `lb` (Load Byte) und `lh` (Load Halfword)
- Bei Byte-Arrays ist das Offset tatsächlich +1 (Element = 1 Byte)

```
char text[] = "ABC";  
char a = text[0];  
char b = text[1]; // Offset in Assembler wirklich +1
```

Notizen

Bei `char`-Arrays gilt: Index 0 → Offset 0, Index 1 → Offset 1. Niemals `lw` für einzelne Zeichen — sonst ein Brei aus vier Buchstaben. Befehl: `lb` (Load Byte).

Load Byte (`lb`)

- Syntax: `lb rd, offset(rs1)`
- Liest genau 1 Byte von der Adresse
- Zielregister `rd` ist 32 Bit groß
- 8 Bits geladen — was passiert mit den restlichen 24 Bits?
- Die Architektur füllt die Lücke automatisch (Sign-Extension)

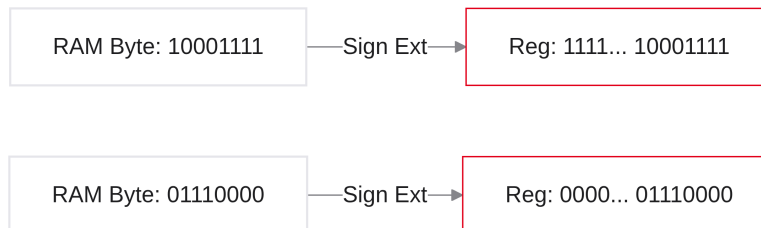
```
lb t0, 0(s1)  
  
# RAM liefert z. B. 0x8F (8 Bits)  
# Register t0: 0x??????8F - linke Bits werden aufgefüllt
```

Notizen

Syntax wie bei `lw`, Hardware anders: Ein Byte (z. B. `0x8F`) liegt rechts im 32-Bit-Register; links klaffen 24 Bits. Die CPU muss sie definieren — und im Zweierkomplement das Vorzeichen korrekt fortsetzen (Sign-Extension). Unsigned-Varianten folgen im nächsten Schritt.

Sign-Extension (Vorzeichenerweiterung)

- Nach `lb` bleiben links im 32-Bit-Register 24 Bits leer
- Die CPU darf diese Lücke nicht mit Zufallswerten oder sturen Nullen füllen
- Lösung: automatische Vorzeichenerweiterung (Sign-Extension)
- Bit 7 des geladenen Bytes wird in alle 24 linken Bits kopiert
- So bleibt der Zweierkomplement-Wert (z. B. -5) identisch



Notizen

Gleiches Problem wie bei `Immediates`: 8 Bits vom RAM, 32-Bit-Register. Wäre das Byte negativ (`MSB = 1`) und würden wir links mit Nullen füllen, entstünde eine große positive Zahl. `lb` kopiert Bit 7 in alle linken Bits — Sign-Extension.

Unsigned Load (`lbu`)

- Manche Bytes sind keine Zweierkomplement-Zahlen
- ASCII oder RGB (0 bis 255) haben kein negatives Vorzeichen
- `MSB = 1` bedeutet dort nur einen hohen Wert (z. B. Rot = 255)
- Befehl: `lbu` (Load Byte Unsigned)
- Deaktiviert Sign-Extension und füllt links immer mit Nullen

```
# Farbwert 0-255 aus dem RAM
lbu t0, 0(s1)

# RAM: 0xFF (255) → t0 = 0x000000FF
```

Notizen

Pixelwert 255 (acht Einsen) mit `lb` geladen: die CPU hält das `MSB` für ein Minuszeichen und zerstört den Wert. Für vorzeichenlose Daten nutzen Sie `lbu` — Zero-Fill statt Sign-Extension.

Halfwords laden (lh, lhu)

- Zwischen Word (32 Bit) und Byte (8 Bit): Halfword = 16 Bit (2 Bytes)
- Anwendung: `short` in C, Audio-Samples (z. B. 16-Bit-WAV)
- `lh`: mit Sign-Extension; `lhu`: Unsigned mit Zero-Fill
- Alignment: Offset durch 2 teilbar (0, 2, 4, 6, ...)

```
lh t0, 2(s1)    # 16 Bit mit Vorzeichenerweiterung
lhu t1, 4(s1)    # 16 Bit mit Zero-Fill
```

Notizen

`lh` holt zwei benachbarte Bytes; Unterscheidung signed/unsigned wie bei Byte-Loads. Offset muss gerade sein — sonst Alignment Fault.

Store Byte & Halfword (sb, sh)

- Gegenstücke zum Laden: `sb` und `sh`
- Syntax wie bei `sw`: `sb rs2, offset(rs1)`
- Nur die untersten 8 bzw. 16 Bits des Registers wandern in den RAM
- Obere Bits werden abgeschnitten und ignoriert

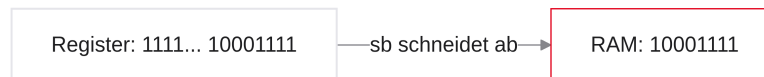
```
# t0 = 0x11223344 - nur 0x44 speichern:
sb t0, 0(s1)
```

Notizen

`sb` schreibt ein ASCII-Zeichen zurück in den String: die CPU schneidet die oberen 24 Bits ab und legt nur die rechten 8 Bits in genau ein RAM-Fach. Der Rest des Speichers bleibt unberührt.

Warum gibt es kein `sbu`?

- Im Handbuch fehlt `sbu` (Store Byte Unsigned) — absichtlich
- Unsigned ist nur relevant, wenn Lücken im Register aufgefüllt werden
- Beim Speichern wird nichts aufgefüllt, nur abgeschnitten
- Dem RAM ist egal, ob die 8 Bits eine Zahl oder ein Pixel sind



Notizen

Trivia mit Tiefgang: `lbu` gibt es, `sbu` nicht. Vorzeichen zählen nur beim Wachsen (Lücke füllen). Beim Quetschen in den RAM reicht Abschneiden — ein `sbu` wäre Hardwareverschwendung.

Decodierung: Der `lw`-Befehl

- `lw` nutzt das bekannte I-Type-Format
- Opcode: Load
- `rs1`: Base-Register
- `imm`: 12-Bit-Offset
- `rd`: Zielregister für den RAM-Wert

Bits	Feld
31:20	<code>imm[11:0]</code> (Offset)
19:15	<code>rs1</code> (Base)
14:12	<code>funct3</code> (Größe)
11:7	<code>rd</code> (Ziel)
6:0	<code>opcode</code> (Load)

Notizen

`lw` passt ins I-Type-Formular wie `addi`: Opcode, Base (`rs1`), 12-Bit-Offset, Ziel (`rd`). Alles in der 32-Bit-Schablone.

Das Problem beim **sw**-Befehl

- **sw** braucht Base (*rs1*), Datenquelle (*rs2*) und 12-Bit-Offset
- Das I-Type-Format hat keinen Platz für *rs2* (dort liegt das Immediate)
- R-Type hat kein Immediate; I-Type hat kein *rs2*
- Für Stores braucht es ein neues Formular

Format	<i>rs1</i>	<i>rs2</i>	<i>rd</i>	Immediate
R-Type	ja	ja	ja	nein
I-Type	ja	nein	ja	ja (12 Bit)
Bedarf sw	ja	ja	nein	ja (12 Bit)

Notizen

Store braucht zwei Quellregister und ein Immediate gleichzeitig — kein bisheriges Format deckt das. Deshalb ein neues Instruction Format.

Das S-Type-Format (Die Lösung)

- Store-Befehle nutzen das S-Type Instruction Format
- Kein Zielregister *rd* (Schreiben geht in den RAM)
- Das 5-Bit-*rd*-Feld wird für *imm[4:0]* wiederverwendet
- *imm[11:5]* liegt links (analog zu *funct7* beim R-Type)
- Das Offset ist im Maschinenwort zweigeteilt

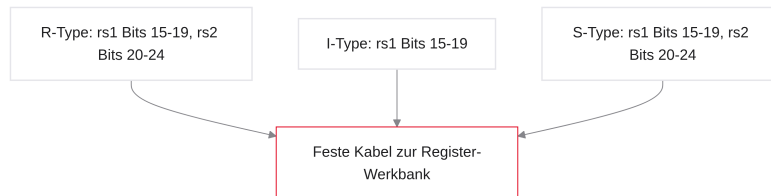
Bits	Feld
31:25	<i>imm[11:5]</i> (oberes Offset)
24:20	<i>rs2</i> (Quelle)
19:15	<i>rs1</i> (Base)
14:12	<i>funct3</i> (Größe)
11:7	<i>imm[4:0]</i> (unteres Offset)
6:0	opcode (Store)

Notizen

Weil Store nichts ins Register File schreibt, ist *rd* frei. RISC-V zerschneidet das 12-Bit-Offset: untere 5 Bits an der *rd*-Position, obere 7 Bits ganz links.

Warum zerrisst RISC-V das Immediate?

- Zerrissenes Immediate wirkt auf den ersten Blick umständlich
- Grund: Hardware-Effizienz — Silizium sparen
- In (fast) jedem Format liegen `rs1` und `rs2` an denselben Bit-Positionen
- Der Kabelbaum zum Register File muss nicht umgeschaltet werden



Notizen

Vergleichen Sie S-Type mit R-Type: `rs1` und `rs2` sitzen fest. Der Decoder verdrahtet einmal; der Compiler zahlt den Preis, das Immediate zu splitten. Typischer RISC-Kompromiss: einfache Hardware, etwas mehr Arbeit in der Toolchain.

Zusammenfassung: Load & Store

- Werkzeuge, um die Memory Wall zu überwinden
- Alle nutzen Base + Offset mit runden Klammern
- Das Offset wird vorzeichenerweitert — auch negative Offsets wie `-4 (s1)` sind möglich

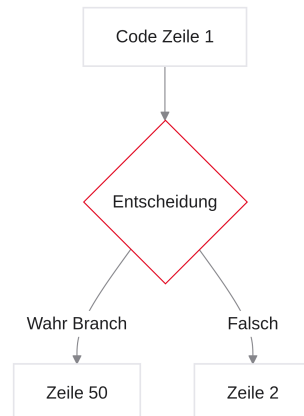
Mnemonic	Name	Breite	Sign-Ext. (Laden)
<code>lw / sw</code>	Word	32 Bit (4 Bytes)	entfällt / (egal)
<code>lh / sh</code>	Halfword	16 Bit (2 Bytes)	ja / (egal)
<code>lhu</code>	Half Unsigned	16 Bit (2 Bytes)	nein (Zero-Fill)
<code>lb / sb</code>	Byte	8 Bit (1 Byte)	ja / (egal)
<code>lbu</code>	Byte Unsigned	8 Bit (1 Byte)	nein (Zero-Fill)

Notizen

Spickzettel für alle Speicherbefehle. Syntax `lw rd, offset(rs1)` prägen. Negative Offsets (12-Bit-Zweierkomplement) erlauben Rückwärtsbewegungen im Array, z. B. Offset `-4`.

Der Kontrollfluss (Branches)

- Ausbruch aus der starren, linearen Code-Ausführung
- Program Counter (PC) und seine Manipulation
- Labels als Ankerpunkte im Text
- Bedingte Sprünge (`beq`, `bne`, `blt`) für If/Else
- Operanden vertauschen, um fehlende Relationen zu simulieren



Notizen

Datenverarbeitung ist geschafft. Nun die Intelligenz des Programms: Entscheidungen. Bisher fiel der Code von oben nach unten. If und Schleifen erfordern Sprünge — die Welt der Branches.

Der Program Counter (Der Dirigent)

- Der PC speichert die RAM-Adresse des aktuellen Befehls (Venus zeigt genau diese Adresse)
- Folgeinstruktion: $PC + 4$; Normalfall: Hardware erhöht nach dem Befehl um +4
- Warum 4? Jeder RV32I-Befehl ist exakt 4 Bytes groß
- Der Prozessor fällt von Zeile zu Zeile (Fall-Through)

RAM-Adresse	Maschinencode	Assembler
0x00000000	0x...	<code>add t0, t1, t2</code> $\leftarrow PC$
0x00000004	0x...	<code>sub t3, t0, t1</code> $\leftarrow PC + 4$
0x00000008	0x...	<code>lw t4, 0(s1)</code> $\leftarrow PC + 4$

Notizen

Kontrollfluss heißt: den PC manipulieren. Standard ist fest verdrahtetes +4. Entscheidungen erfordern, diesen Rhythmus zu durchbrechen und den PC gezielt zu überschreiben.

Das IF-Problem

- In C ist Kontrollfluss abstrahiert: `if (a == b) { ... } else { ... }`
- Die CPU kennt keine geschweiften Klammern
- Zum Überspringen muss der PC auf die Adresse nach dem Block zeigen
- Adressabstände per Hand zu pflegen wäre fehleranfällig

```
if (a == b) {  
    // Block ausführen  
}  
// Wie finden wir diese Zeile im RAM?
```

Notizen

Geschweifte Klammern sind Kosmetik. Im Maschinencode liegen Befehle hintereinander. Bei falscher Bedingung muss die CPU wissen, wie viele Bytes sie überspringt. Labels nehmen uns die Byte-Rechnung ab.

Labels (Sprungmarken)

- Labels: Textmarken mit Doppelpunkt (:) im Quelltext
- Keine Prozessor-Befehle — kosten keine Ausführungszeit
- Der Assembler berechnet die exakte RAM-Adresse des Labels
- Sprungbefehle referenzieren den Label-Namen, nicht die Hex-Adresse

```
add t0, t1, t2  
ZielMarke:          # Label → z. B. Adresse 0x08  
sub t3, t0, t1
```

Notizen

Beim Assemblieren scannt Venus die Labels und ersetzt Namen durch die korrekte Distanz bzw. Adresse. Labels sind Ihr Navi im Code.

Branch Equal (beq)

- Syntax: `beq rs1, rs2, Label` (Branch if Equal)
- Vergleich der Registerinhalte
- Bei Gleichheit: `PC → Label-Adresse` (Sprung)
- Bei Ungleichheit: Fall-Through mit `PC +4`

```
beq t0, t1, Weiter
addi t2, t2, 1      # übersprungen, wenn t0 == t1
Weiter:
sub t3, t0, t1
```

Notizen

`beq`: ALU vergleicht (bzw. prüft Differenz auf Null). Gleich → Sprung zum Label. Ungleich → nächste Zeile. Hinweis: Im Beispiel `addi` statt `add` mit Immediate — konsistent zur RV32I-Syntax.

Branch Not Equal (bne)

- Gegenstück zu `beq`: `bne rs1, rs2, Label`
- Springt nur bei unterschiedlichen Registerwerten
- Typisches Schleifen-Muster: solange Zähler ungleich Grenze, zurück nach oben

```
li    t1, 10          # Grenze vor der Schleife
LoopStart:
# ... Schleifen-Rumpf ...
addi t0, t0, 1
bne t0, t1, LoopStart  # zurück, solange t0 != 10
```

Notizen

`bne` ist das Standardwerkzeug für Schleifen: Zähler erhöhen, mit Grenze vergleichen, bei Ungleichheit zurückspringen. Bei Gleichheit endet die Schleife (Fall-Through).

Relationen (b1t, bge)

- b1t: Branch if Less Than — springt wenn $rs1 < rs2$
- bge: Branch if Greater or Equal — springt wenn $rs1 \geq rs2$
- Interpretation als Zweierkomplement (mit Vorzeichen)
- -5 ist für b1t korrekt kleiner als 2

```
# Sensorwert t0 kleiner als Grenzwert t1?
b1t t0, t1, AlarmAusloesen
```

Notizen

Gleichheit reicht oft nicht. b1t und bge nutzen vorzeichenbehaftete Arithmetik — negative Werte werden korrekt eingeordnet.

Unsigned Branches (b1tu, bgeu)

- Pointer und Rohdaten (Farben) sind oft vorzeichenlos
- Adresse `0xF0000000` wäre als signed negativ — b1t läge falsch
- Suffix u: b1tu, bgeu vergleichen rein positiv (ohne Vorzeichen)

```
b1tu t0, t1, AdresseIstKleiner
```

Notizen

Array-Ende per Pointer vergleichen: bei großen Adressen zwingend b1tu/bgeu. Sonst hält die ALU das MSB für ein Minuszeichen und trifft die falsche Entscheidung.

Warum gibt es kein $\leq$?

- Keine Hardware-Befehle für $\leq$ oder $>$
- Absichtlicher Minimalismus: jeder Befehl kostet Decoder-Fläche
- $A \leq B$ ist logisch identisch mit $B \geq A$
- Fehlendes $\leq$ simulieren: Operanden tauschen und bge nutzen

```
# Gewünscht: if (t0 <= t1)
# Identisch: if (t1 >= t0)
bge t1, t0, Label
```


Notizen

RISC-V spart Transistoren: statt neuer Relationen tauschen Sie die Operanden. Dieselbe Logik, kleinere Hardware.

Folie

50 / 72

Pseudo-Banches (**ble**, **bgt**)

- Ständiges Umdenken der Operanden ist fehleranfällig
- Venus bietet Pseudo-Instruktionen: z. B. `ble` (Branch Less Equal)
- Der Assembler vertauscht Operanden und erzeugt echtes `bge`

```
# Lesbar im Quelltext:  
ble t0, t1, Label  
  
# Tatsächliche Hardware:  
bge t1, t0, Label
```

Notizen

Wie bei `li`: Pseudo-Instruktionen erhöhen Lesbarkeit. Der Chip kennt `ble` nicht; Venus übersetzt es in ein hardwarekonformes `bge`.

Folie

51 / 72

Architektur: Das B-Type-Format (Vorschau)

- Branches nutzen das B-Type-Format (Vorschau; vereinfacht: verwandt mit S-Type)
- Das Label wird zum Immediate (Distanz), nicht zur absoluten RAM-Adresse
- Neuer PC = alter PC + Immediate (PC-relative Adressierung)
- Vorteil: verschiebbarer Code bleibt korrekt

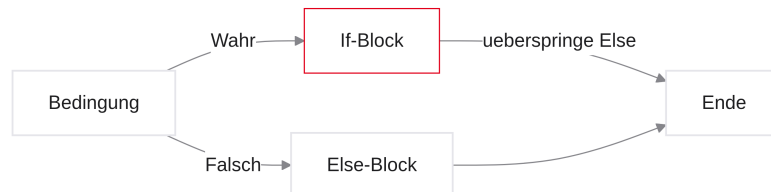
```
Warum relativ?  
Absolute Adressen brechen, wenn das Programm im RAM verschoben wird.  
Die relative Distanz (z. B. +16 Bytes) bleibt gleich.
```

Notizen

PC-relative Sprünge: "gehe 20 Bytes nach unten" funktioniert auch, wenn das OS das Programm woanders lädt. Absolute Adressen würden crashen. Hinweis zur Vereinfachung: B-Type teilt sich mit S-Type die Idee getrennter Immediate-Felder, das konkrete Bit-Layout weicht jedoch ab — Details bei Bedarf später.

Unbedingte Sprünge (Jumps)

- Architektonisches Puzzleteil für vollständige Kontrollstrukturen
- Warum Branches allein nicht ausreichen
- Code-Blöcke ohne Wenn und Aber überspringen
- Pseudo-Instruktion `j` (Jump)
- Mentaler Algorithmus: If/Else aus C übersetzen



Notizen

Branches treffen Entscheidungen. Für If/Else reicht das nicht: Nach dem If-Block darf der Else-Block nicht versehentlich mitlaufen. Dafür brauchen wir bedingungslose Umleitungen — Jumps.

Das If-Else-Problem

- If/Else erzwingt eine Entweder-Oder-Entscheidung
- Beide Blöcke liegen im RAM direkt untereinander
- Bei wahrer Bedingung: Fall-Through durch den If-Block
- Danach fällt der PC physikalisch in den Else-Block
- Am Ende des If-Blocks muss der Else-Block strikt übersprungen werden

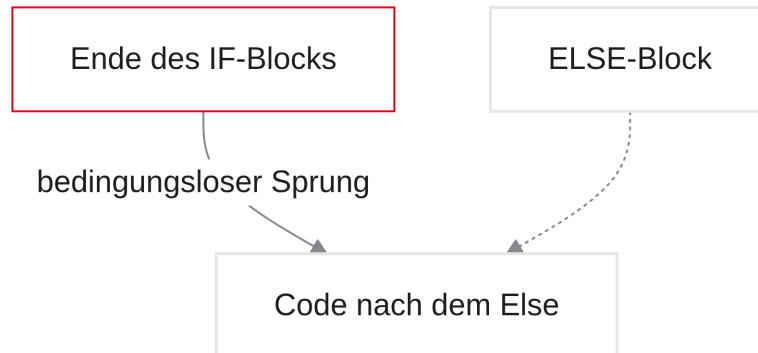
```
if (a == b) {  
    // wahr: nur dieser Block ...  
    // ... dann SOFORT das gesamte Konstrukt verlassen  
} else {  
    // falsch: nur dieser Block  
}
```

Notizen

In C: entweder If oder Else. In Assembler liegen beide Blöcke sequenziell. Ohne Gegenmaßnahme läuft der PC nach dem If in den Else. Am If-Ende brauchen wir: `springe bedingungslos ans Ende`.

Der bedingungslose Sprung

- Branches (`beq`, `blt`) fragen die ALU nach einer Bedingung
- Wir brauchen: Springe. Immer.
- Solche Befehle heißen Jumps
- Sie überschreiben den PC absolut bedingungslos
- Unverzichtbar für If-Ausstiege, Endlosschleifen und später Funktionsaufrufe



Notizen

Kein Branch: wir vergleichen keine Register, wir brechen aus. Ein Jump nimmt ein Label und setzt den PC sofort. Assembler-Handwerkszeug zum Verknüpfen von Code-Bausteinen — analog zu frühem `goto`.

Jump (`j`) — die Pseudo-Instruktion

- Im Labor: Pseudo-Instruktion `j`
- Syntax: `j Label`
- Label → relatives Offset; die CPU addiert es zum PC
- Code physisch unter dem `j` wird nicht erreicht (toter Code)

```

add t0, t1, t2
j Ueberspringen      # sofort zum Label

sub t3, t0, t1       # wird nie ausgeführt

Ueberspringen:
  lw t4, 0(s1)
  
```

Notizen

`j` kennt nur ein Label — keine Operanden-Register. Der Befehl darunter ist unerreichbar, es sei denn, ein anderer Sprung zielt explizit darauf.

Vorschau: Der echte Befehl (jal)

- Der Chip kennt `j` nicht; Venus übersetzt zu `jal`
- `jal` = Jump and Link
- Syntax: `jal rd, Label` — springt und speichert die Rücksprungadresse in `rd`
- Für normales `j` ohne Rückkehr: Ziel `x0` (Hardwired Zero)

```
# Was wir tippen:  
j EndIf  
  
# Was Venus daraus macht (x0-Trick):  
jal x0, EndIf  
# Rücksprungadresse landet wirkungslos in x0
```

Notizen

`jal` ist eigentlich für Funktionsaufrufe (Link = Rückkehradresse). Beim If/Else-Überspringen wollen wir nicht zurück — Venus wirft die Adresse nach `x0`. Mehr zu `jal` in Einheit 4.

If-Else übersetzen (Konzept)

- Beispiel: `if (s0 == s1) { t0 = 1; } else { t0 = 2; }`
- Drei Bausteine: bedingter Sprung (`bne`), unbedingter Sprung (`j`), zwei Labels
- Struktur als Blöcke denken
- Streng in vier algorithmischen Schritten vorgehen

```
if (s0 == s1) {  
    t0 = 1;  
} else {  
    t0 = 2;  
}
```

Notizen

Standard-Klausur- und Labor-Muster. Ohne geschweifte Klammern muss der sequenzielle Speicher streng logisch organisiert werden. Der entscheidende Paradigmenwechsel folgt im nächsten Schritt.

Schritt 1: Die Invertierung

- Wir fragen nicht, ob die C-Bedingung wahr ist
- Wir fragen, ob sie falsch ist
- C: `if (s0 == s1) → Assembler: bne s0, s1, ElseBlock`
- Wahr → Fall-Through in den If-Block (spart Befehle)
- Falsch → Sprung zum Else-Label

```
# Wenn s0 UNGLEICH s1: sofort zum Else
bne s0, s1, ElseBlock

# darunter geht es weiter, wenn sie GLEICH waren
```

Notizen

Einsteigerfehler: C-Logik 1:1. In Assembler invertieren: bei Ungleichheit weg zum Else; bei Gleichheit fällt der PC in den If-Block darunter.

Schritt 2: Der IF-Block

- `bne` hat nicht gegriffen → Bedingung war wahr
- Hier der normale If-Code; kein If-Label nötig (Fall-Through)
- Beispiel: Konstante 1 nach `t0`

```
bne s0, s1, ElseBlock

# IF-Block (erreicht, wenn s0 == s1)
li t0, 1
```

Notizen

Direkt unter dem Branch steht der If-Code. Kein separates If-Label — der PC liegt bereits auf dem richtigen Weg.

Schritt 3: Der rettende Jump

- Unter dem If-Block liegt im Speicher der Else-Block
- Ohne Gegenmaßnahme würde der PC auch `t0 = 2` ausführen
- Am If-Ende: `j EndIf` — Else überspringen
- `EndIf` steht ganz am Ende der Struktur

```
bne s0, s1, ElseBlock

li t0, 1
j EndIf                # Else überspringen
```

Notizen

Häufigster Labor-Fehler: den Jump vergessen. Dann überschreibt der Else-Block das Ergebnis. `j EndIf` rettet den Kontrollfluss.

Schritt 4: Der Else-Block

- Label `ElseBlock`: — Ziel des ersten Branches
- Alternative: `t0 = 2`
- Darunter `EndIf`: als Sammelbecken beider Pfade
- Ab dort wieder linearer Ablauf

```
bne s0, s1, ElseBlock

li t0, 1
j EndIf

ElseBlock:
    li t0, 2

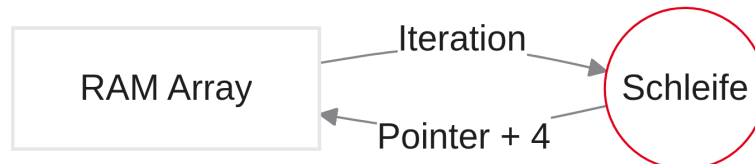
EndIf:
    # ab hier wieder gemeinsam weiter
```

Notizen

Beide Pfade vereinen sich bei `EndIf`: Jump aus dem If oder Fall-Through aus dem Else. Das ist die hardwarenahe Anatomie von If/Else.

Praxis: Algorithmen in Venus

- Transfer in die Labor-Umgebung
- C-Schleifen (`while`, `for`) in Assembler
- Kontrollfluss + Speicherzugriff kombinieren
- Iteration über Arrays im RAM
- Memory-Interface in Venus



Notizen

Theorie-Werkzeuge beisammen. Schleifen = rückwärts gerichteter Kontrollfluss plus Bedingung. Als Nächstes: For/While und Pointer-Bewegung durch Arrays. Heute arbeiten wir in Venus; den visuellen Datenpfad und die Pipeline betrachten wir später in Ripes.

Die `while`-Schleife

- Einfachstes Loop-Konstrukt; kopfgesteuert
- Label am Start (`LoopStart`)
- Branch bei falscher Bedingung zum Exit (`LoopExit`)
- Am Rumpf-Ende: `j LoopStart`

```
int a = 0;
while (a < 10) {
    a++;
}
```

Notizen

Oben prüfen, dann Rumpf, dann zurück. Invertierter Exit-Check + Jump nach oben — dasselbe Muster wie bei `if`.

while in Assembler

- Zähler a in $t0$, Grenze 10 in $t1$
- C: $a < 10 \rightarrow$ Exit mit `bge` (sobald $a \geq 10$)

```
li t0, 0
li t1, 10

LoopStart:
    bge t0, t1, LoopExit    # a >= 10 → raus

    addi t0, t0, 1          # a++

    j LoopStart

LoopExit:
```

Notizen

Muster: Init, Label, invertierter Exit-Branch, Body, Jump zurück. `bge` spiegelt die Umkehrung von $a < 10$.

Die for-Schleife

- Syntaktisch kompakte `while`-Schleife
- Vier Zonen: Init, Condition, Step, Body
- Init vor der Schleife; Step zwingend am Ende des Bodies (vor dem Jump)

```
for ( Init ; Condition ; Step ) {
    Body
}
```

Notizen

Init nur einmal — oberhalb des Start-Labels. `i++` erst nach dem Body, sonst Off-by-One (erster Durchlauf schon mit Index 1).

for in Assembler

- Zähler `i` in `t0`, Grenze in `t1`
- Struktur fast wie `while` — Position des Steps beachten

```
li t0, 0          # INIT
li t1, 10

ForStart:
    bge t0, t1, ForExit    # CONDITION

    # BODY

    addi t0, t0, 1          # STEP (am Ende!)
    j ForStart

ForExit:
```

Notizen

Step vor dem Body → klassischer Off-by-One. Step immer direkt vor dem Jump zurück.

Arrays iterieren (Pointer Math)

- Ziel: alle Array-Elemente aufsummieren
- In der Schleife: `lw`
- Problem: Offset bei `lw` ist eine feste Konstante — keine Variable `i`
- Lösung: Base-Pointer in jedem Durchlauf verschieben

```
summe = summe + array[i]; // i aendert sich dynamisch
```

Notizen

In C ist `i` dynamisch. In Assembler geht `lw t0, i(s1)` nicht. Verschiedene Fächer lesen heißt: den Pointer bewegen.

Das Pointer-Update (Labor-Trick)

- Offset starr auf 0: immer `lw t2, 0(s1)`
- Nächstes Element: Base-Pointer aktualisieren
- Bei 32-Bit-Integern: `addi s1, s1, 4` pro Durchlauf

```
lw t2, 0(s1)           # Wert an aktueller Adresse
add t4, t4, t2          # aufsummieren

addi s1, s1, 4          # Pointer + 4 Bytes
addi t0, t0, 1          # i++
```

Notizen

Offset 0, Finger weiterbewegen. Word-Arrays: immer +4, nicht +1 — sonst Byte-Salat.

Venus Onboarding: Memory-Tab

- Register-Ansicht reicht nicht mehr: Blick in den RAM
- Venus: Reiter Memory (neben Editor/Simulator)
- Hex-Dump; Adressen im Raster `0x00`, `0x04`, `0x08`, ...
- Word-Inspektion: Little-Endian beachten

Notizen

Memory-Tab: Adresse links, Bytes in der Mitte — Ergebnis Ihrer `sw`-Befehle. Ideal zum Prüfen, ob das Array sauber liegt.

Daten im Editor deklarieren (`.data`)

- Startwerte vor Programmstart in den RAM bringen
- Direktive `.data` für statische Variablen
- `.word` legt 32-Bit-Werte hintereinander (Abstand 4 Bytes)
- Im `.text`-Block: Adresse per Label laden (`la`)

```

.data
myArray: .word 10, 20, 30, 40, 50

.text
.globl main
main:
    la s1, myArray    # Load Address - Pointer nach s1

```

Notizen

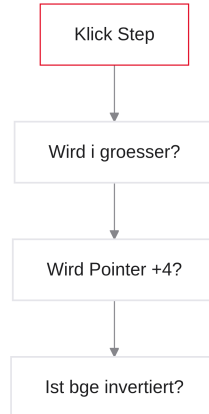
Unter `.data` legt der Assembler das Array vor dem Start an. `la` (Load Address) holt die Hausnummer nach `s1`. Programmcode steht unter `.text`.

Folie

71/72

Debugging von Schleifen

- Falscher Branch → Endlosschleife (Run stoppt nicht)
- Workflow: Register auf Dezimal, nur Step nutzen
- Prüfen: wächst der Zähler (`t0`)?
- Prüfen: springt der Pointer (`s1`) in 4er-Schritten?
- Prüfen: löst `bge` an der Grenze korrekt aus?



Notizen

Kein Panikmodus: steppen. Pro Durchlauf müssen Zähler und Pointer wachsen. Bleibt eines stehen — fehlendes `addi` oder falsches Register.

Labor-Aufgabe: Array summieren

- Mit einer Schleife über ein Array iterieren und die Summe bilden
- Setup: Array mit 5 Werten unter `.data`
- Ziel: Endergebnis in `a0`

Checkliste:

- Array-Adresse laden (`la`)
- Startwerte: Summe = 0, Zähler = 0, Grenze = 5
- Labels und Exit-Branch (`bge`)
- Wert lesen (`lw`) und zur Summe addieren
- Pointer um 4 erhöhen
- Zähler um 1 erhöhen

```
# Viel Erfolg im Venus-Simulator!
```

Notizen

For-Gerüst wie auf den Folien: `lw`, aufaddieren, am Ende Zähler +1 und Pointer +4. Liegt die Summe in `a0`, sind Speicher und Kontrollfluss verstanden. Viel Erfolg im Hands-on!